

Tag 3

Dateien & Fehlerbehandlung

Dateien lesen, speichern und Fehler kontrolliert behandeln

Was du heute lernst

1. Wie **Dateien geöffnet und gelesen** werden
2. Wie Dateien **geschrieben und erweitert** werden
3. Warum `with` beim Arbeiten mit Dateien wichtig ist
4. Wie `try` / `except` Fehler abfangen
5. Wie `raise` eigene Fehler auslöst
6. Wie **CSV und JSON** gespeichert und geladen werden
7. Wie `pathlib` Dateipfade verwaltet

Block 1

Dateien

Eine Datei öffnen

Zum Öffnen verwenden wir `open()` .

```
datei = open(  
    "notizen.txt",  
    encoding="utf-8"  
)
```

Damit haben wir Zugriff auf die Datei.

Dateien mit `with` öffnen

Besser ist die Verwendung von `with`.

```
with open(
    "notizen.txt",
    encoding="utf-8"
) as datei:
    inhalt = datei.read()

print(inhalt)
```

`with` sorgt dafür,
dass die Datei anschließend
automatisch geschlossen wird.

Dateien lesen

Mit `.read()` lesen wir
den kompletten Inhalt.

```
with open(  
    "notizen.txt",  
    encoding="utf-8"  
) as datei:  
    inhalt = datei.read()  
  
print(inhalt)
```

Die Variable `inhalt`

enthält danach den Text der Datei.

Zeilenweise lesen

Eine Datei kann auch

Zeile für Zeile durchlaufen werden.

```
with open(  
    "notizen.txt",  
    encoding="utf-8"  
) as datei:  
  
    for zeile in datei:  
        print(zeile)
```

Das ist praktisch,

wenn eine Datei viele Zeilen enthält.

Der Dateimodus

Beim Öffnen kann angegeben werden,
was mit der Datei passieren soll.

```
"r" → lesen  
"w" → schreiben  
"a" → anhängen
```

Der Modus wird an `open()` übergeben.

```
open("notizen.txt", "r")
```

Schreiben mit "w"

Mit "w" schreiben wir
neuen Inhalt in eine Datei.

```
with open(  
    "notizen.txt",  
    "w",  
    encoding="utf-8"  
) as datei:  
  
    datei.write("Erster Eintrag")
```

Achtung:

"w" überschreibt vorhandenen Inhalt.

Anhängen mit "a"

Mit "a" wird neuer Inhalt
am Ende hinzugefügt.

```
with open(  
    "notizen.txt",  
    "a",  
    encoding="utf-8"  
) as datei:  
    datei.write("\nZweiter Eintrag")
```

Vorhandener Inhalt bleibt erhalten.

Die drei wichtigsten Modi

"r"

→ Datei lesen

"w"

→ Datei schreiben

→ vorhandener Inhalt wird überschrieben

"a"

→ Inhalt anhängen

Block 2

Fehlerbehandlung

Was passiert bei einem Fehler?

Was passiert hier?

```
with open(  
    "geheim.txt",  
    encoding="utf-8"  
) as datei:  
  
    inhalt = datei.read()
```

Existiert die Datei nicht,

entsteht ein Fehler:

```
FileNotFoundError
```

try und except

Mit `try` können wir Code ausführen,
bei dem ein Fehler auftreten kann.

```
try:
    with open(
        "geheim.txt",
        encoding="utf-8"
    ) as datei:
        inhalt = datei.read()

except FileNotFoundError:
    print("Datei nicht gefunden.")
```

`try` → Versuch

`except` → Fehler behandeln

Eingaben können ebenfalls fehlschlagen

Zum Beispiel:

```
zahl = int(  
    input("Zahl: ")  
)
```

Bei:

hallo

entsteht:

ValueError

ValueError behandeln

```
try:
    zahl = int(
        input("Zahl: ")
    )

except ValueError:
    print("Keine gültige Zahl.")
```

Das Programm kann den Fehler
kontrolliert behandeln.

Nicht jeden Fehler verschlucken

Besser:

```
except ValueError:  
    ...
```

als:

```
except:  
    ...
```

Je genauer der abgefangene Fehler,
desto besser können wir reagieren.

else

Nach einem erfolgreichen `try`
kann `else` ausgeführt werden.

```
try:  
    zahl = int(input("Zahl: "))  
  
except ValueError:  
    print("Ungültig.")  
  
else:  
    print("Eingabe war korrekt.")
```

`else` läuft also nur,

wenn kein Fehler aufgetreten ist.

finally

`finally` wird immer ausgeführt.

```
try:  
    zahl = int(input("Zahl: "))  
  
except ValueError:  
    print("Ungültig.")  
  
finally:  
    print("Programm beendet.")
```

`finally` eignet sich für Code,
der unabhängig vom Ergebnis ausgeführt werden soll.

Block 3

Eigene Fehler

raise

Manchmal ist eine Eingabe zwar technisch gültig,
aber inhaltlich nicht erlaubt.

```
alter = int(  
    input("Alter: ")  
)  
  
if alter < 0:  
    raise ValueError(  
        "Alter darf nicht negativ sein."  
    )
```

Mit `raise` lösen wir

bewusst einen Fehler aus.

Validierung

`raise` eignet sich deshalb

für eigene Regeln.

```
betrag = float(  
    input("Betrag: ")  
)  
  
if betrag < 0:  
    raise ValueError(  
        "Betrag darf nicht negativ sein."  
    )
```

`input()` prüft nur,

ob etwas eingegeben wurde.

Unsere Logik prüft,

Fehlerbehandlung zusammen

```
try:
    alter = int(
        input("Alter: ")
    )

    if alter < 0:
        raise ValueError(
            "Alter darf nicht negativ sein."
        )

except ValueError as fehler:
    print("Fehler:", fehler)
```

Hier verbinden wir:

```
try
↓
Eingabe
↓
ValueError
```

Block 4

CSV

CSV mit Python lesen

Python besitzt dafür

das Modul `csv`.

```
import csv

with open(
    "personen.csv",
    encoding="utf-8"
) as datei:

    reader = csv.DictReader(datei)

    for person in reader:
        print(person["Name"])
```

Was macht DictReader?

Aus einer Zeile wie:

```
Anna,32,Bielefeld
```

wird sinngemäß:

```
{  
    "Name": "Anna",  
    "Alter": "32",  
    "Ort": "Bielefeld"  
}
```

CSV → strukturierte Python-Daten

Block 5

JSON

Mit `json.dump()` schreiben wir

Python-Daten in eine Datei.

```
import json

person = {
    "name": "Anna",
    "alter": 32,
    "skills": [
        "Python",
        "Linux"
    ]
}

with open(
    "person.json",
    "w",
    encoding="utf-8"
) as datei:

    json.dump(
        person,
        datei,
        indent=4
    )
```

JSON laden

Mit `json.load()` lesen wir

JSON wieder ein.

```
import json

with open(
    "person.json",
    encoding="utf-8"
) as datei:

    person = json.load(datei)

print(person["name"])
print(person["skills"])
```

Danach haben wir wieder
eine Python-Datenstruktur.

CSV oder JSON?

CSV

- Tabellen
- Zeilen und Spalten
- einfache Datensätze

JSON

- strukturierte Daten
- Dictionaries und Listen
- verschachtelte Daten

Block 6

`pathlib`

Dateipfade mit `pathlib`

Für Dateipfade gibt es
das Modul `pathlib`.

```
from pathlib import Path  
datei = Path("daten") / "notizen.txt"
```

Der Pfad wird dabei
plattformunabhängig zusammengesetzt.

Datei mit `pathlib` prüfen

```
from pathlib import Path

datei = Path("notizen.txt")

if datei.exists():
    print("Datei vorhanden.")
```

Auch Verzeichnisse können
über `Path` verwaltet werden.

Lesen und Schreiben

`pathlib` kann einfache Dateien

direkt lesen und schreiben.

```
from pathlib import Path

datei = Path("notizen.txt")

datei.write_text(
    "Hallo Python!",
    encoding="utf-8"
)

inhalt = datei.read_text(
    encoding="utf-8"
)

print(inhalt)
```

`open()` oder `pathlib`?

Beides ist sinnvoll.

`open()`

- klassische Dateiverarbeitung
- gut für `csv`, `json` und Schleifen

`pathlib`

- moderne Verwaltung von Pfaden
- Dateien und Verzeichnisse